

DeepScientist Citation Audit Agent: Project Report

Course: Final Project for *Large Language Models: From Principles to Applications*

Track: Project A, Scientific Agent Function Extension

Base System: Official DeepScientist + Claude Code

Team Members: Guo Yaoyuan, Qi Jianan, Liu Yannan

Abstract

This project follows Project A and extends a scientific research agent. We use the official DeepScientist system as the base research agent. DeepScientist reads an input PDF, understands the paper, searches relevant literature, and generates research ideas or hypotheses. Our project adds a verifiable Citation Evidence Chain Tracking Module after hypothesis generation and before final report writing. The module checks whether the literature cited by the generated idea contains citation errors and outputs a Green / Yellow / Red citation audit table.

The goal is not to replace DeepScientist's idea-generation ability. Instead, the goal is to address a common citation hallucination problem in scientific agents: a cited paper may look real but have a nonexistent DOI, its metadata may not match public records, or the paper may exist but fail to support the claim. To satisfy the course requirement that the system must not rely only on prompts and must demonstrate tool use, this project implements self-written DeepScientist skills, a self-written citation verifier, an evidence chain tracer, public scholarly API retrieval, tool-call logging, QQ live interaction, and reproducible output directories.

1. Project Title and Track

The project title is **DeepScientist Citation Audit Agent**. The selected track is Project A: adding an evidence-chain tracking module to a scientific research agent.

The instructor's updated requirement states that the input should be PDFs from different disciplines. The agent should search the literature like a human researcher and form a new research idea or hypothesis. The system should then check whether the references used by that generated idea contain errors. If a cited paper correctly supports the claim, it should be labeled Green. If the support relation is uncertain, it should be labeled Yellow. If the citation is definitely wrong, it should be labeled Red.

Therefore, the task of this project can be summarized as:

```
PDF paper input
-> Official DeepScientist generates hypotheses and citation-backed claims
-> Self-written citation audit module verifies the cited literature
-> Green / Yellow / Red evidence-chain report
```

DeepScientist is the base scientific agent. The new contribution of this project is the citation audit and evidence-chain tracking module.

2. Base System

This project uses the official DeepScientist system as the base system and uses Claude Code as the DeepScientist runner. DeepScientist provides the core scientific-agent workflow:

```
read papers
-> identify research gaps
-> generate hypotheses
-> prepare a research report
```

Our added module is inserted after hypothesis generation and before final report writing:

```
Official DeepScientist quest
-> citation_audit_claims.json
-> citation_evidence_chain_audit
-> citation_verification.csv
-> evidence_chain.csv
-> final audit report
```

To make the official DeepScientist workflow compatible with this project, we implemented two main skills:

1. `citation-hypothesis-claims`

This skill asks DeepScientist to generate idea cards and citation-backed claims from a PDF, and to write `citation\_audit\_claims.json`.

2. `citation-evidence-audit`

This skill sends `citation\_audit\_claims.json` to the local deterministic verifier and produces Green / Yellow / Red audit results and evidence chains.

The key boundary is that DeepScientist generates scientific ideas, while this project verifies whether the citations behind those ideas are reliable. Green / Yellow / Red labels are not directly decided by an LLM. They are produced by local rules, public scholarly APIs, and evidence-chain records.

3. Problem Definition

When a scientific agent generates a research hypothesis or paper analysis, it often cites papers that appear professional. However, such citations may contain several types of errors:

- ? The cited paper does not exist, or the DOI/title is fabricated.
- ? The paper exists, but the author, year, title, or DOI does not match public records.
- ? The paper exists and its metadata is correct, but its abstract or retrieval snippet does not support the claim.
- ? The claim is too strong, such as turning "possibly related" into "proven".
- ? The version is inconsistent, such as an arXiv preprint and a journal version having different years.
- ? An API query fails or no abstract is available, making the support relation impossible to confirm.

If we only ask a large language model to judge these problems, the system may become "using hallucination to audit hallucination." Therefore, this project decomposes citation verification into three interpretable layers:

1. Existence check: whether the cited work exists;
2. Metadata check: whether the title, author, year, and DOI match;
3. Support check: whether the abstract, snippet, or supporting sentence supports the claim.

The final color-labeling rules are:

- ? Green: the cited paper exists, metadata matches, and a concrete supporting sentence is found.

? Yellow: the cited paper exists, but support is uncertain, incomplete, only partial, or requires manual review.

? Red: the cited paper is missing, has clear metadata errors, or available evidence clearly does not support the claim.

4. System Design

The system consists of five major components.

4.1 Official DeepScientist Integration

The system uses `scripts/run_official_ds_case.py` and campaign runners to create or reuse DeepScientist quests. It provides a fixed prompt to DeepScientist and asks it to generate `citation_audit_claims.json`. This JSON file is the interface boundary between official DeepScientist and this project's audit module.

4.2 Citation Audit Toolchain

The local citation audit toolchain reads `citation_audit_claims.json` and performs public-API retrieval and rule-based verification for each cited work. The main providers are:

- ? Crossref;
- ? OpenAlex;
- ? Semantic Scholar;
- ? arXiv.

Every provider query is written to `tool_calls.jsonl`; failed API calls are not silently ignored. API failures, timeouts, and 429 rate limits are recorded and usually lead to Yellow rather than fabricated Green results.

4.3 Evidence Chain Tracing

The system assigns an evidence ID or tool-call ID to each claim, each tool call, and each retrieved evidence item. The final report can trace:

```
claim
-> cited work
-> provider lookup result
-> metadata/support status
-> evidence id
-> Green / Yellow / Red label
```

This allows the instructor to directly inspect `citation_verification.csv`, `evidence_chain.csv`, `tool_calls.jsonl`, and `evidence_items.jsonl` to understand how the system reached each result.

4.4 QQ Bot Interaction

To satisfy the live demonstration requirement, the system provides a QQ Bot interaction layer. The main commands are:

```
/official "C:\path\paper.pdf"
/local "C:\path\paper.pdf"
/help
```

The `/official` command starts the official DeepScientist workflow and then runs the citation audit after claims are generated. The QQ interaction returns progress messages, the run directory, Green / Yellow / Red counts, the multi-reviewer decision, and paths to the key artifacts.

4.5 Multi-reviewer and Memory

The Multi-reviewer module is mainly implemented by Guo Yaoyuan, and the Memory module is mainly implemented by Qi Jianan. They operate as a quality-control layer after the citation audit. The multi-reviewer module scores idea quality and demonstration risk, while the memory module records historical citations, provider behavior, failures, and reviewer decisions. Importantly, neither the multi-reviewer nor memory module directly overrides Green / Yellow / Red labels. The final labels remain determined by the deterministic verifier.

5. Test Cases and Results

The project has been tested through multiple local workflows, official DeepScientist campaigns, and QQ live interaction runs. The statistics below are collected from:

```
outputs/runs/  
outputs/deepscientist_15x_campaigns/  
outputs/deepscientist_20x_campaigns/
```

The current statistics are:

Metric	Value
Citation audit CSV files	33
Checked claims	201
Green	61
Yellow	138
Red	2
Tool-call rows	1408
Evidence items	985
Multi-review reports	24

These numbers show that the system was not tested on only one demonstration case. Instead, it preserved evidence artifacts from many real runs. The large number of Yellow labels is expected because the system uses a conservative audit policy: if concrete support cannot be confirmed, the citation is not promoted to Green.

5.1 Success Case: CLIP

The success case is `qq\_pdf\_ai\_clip\_transfer`, with the run directory:

```
outputs/deepscientist_20x_campaigns/  
qq_official_20260605_011710/  
cases/qq_pdf_ai_clip_transfer/
```

The audit result is:

```
Green = 5  
Yellow = 1  
Red = 0
```

A representative Green claim states that visual models are typically trained to predict fixed categories. The system cites the CLIP paper, `Learning Transferable Visual Models From Natural Language Supervision`, and finds the supporting sentence:

```
State-of-the-art computer vision systems are trained to predict a fixed set of predetermined object categories.
```

This citation satisfies the Green criteria: the paper exists, metadata matches, and the supporting sentence directly supports the claim.

5.2 Medical Case: CheXNet

For the QQ live interaction case, we use a medical AI PDF related to CheXNet. This case demonstrates that the system can receive a PDF path through a QQ command, start the official DeepScientist workflow, continuously return progress heartbeat messages, and finally generate the citation audit result. The workflow outputs the run directory, Green / Yellow / Red counts, multi-reviewer score, and paths to the key artifacts.

5.3 Cross-domain Testing

The project also tested public papers from AI, medicine, physics, optimization, education, and finance. Example cases include Transformer, RAG, CheXNet, CLIP, Adam optimizer, and GW150914. These cases test whether the system can handle PDFs from different disciplines and citation metadata with different levels of completeness.

6. Failure and Boundary Cases

The boundary case is `qq\_pdf\_physics\_gravitational\_waves`, with the run directory:

```
outputs/deepscientist_20x_campaigns/  
qq_official_20260605_035133/  
cases/qq_pdf_physics_gravitational_waves/
```

The result is:

```
Green = 3  
Yellow = 2  
Red = 1
```

This is not the simplest type of fake-DOI error. It is closer to the type of problem that may occur in real citation audits: a paper may exist, but its metadata or support relation may still be risky. The system found that one citation had an author metadata mismatch or support mismatch, so it was labeled Red or required strict review.

Observed failure and boundary cases include:

- ? Official DeepScientist may take a long time to generate `citation\_audit\_claims.json`.
- ? Semantic Scholar or arXiv may return 429 errors or timeouts.
- ? A paper may exist but have no available abstract.
- ? arXiv and journal versions may have different years.
- ? A claim may be too strong while public snippets only partially support it.
- ? Different providers may return inconsistent metadata.

The system handles these cases conservatively: API failures are not fabricated as success; missing abstracts are not forced into Green; version or year uncertainty becomes Yellow; clear metadata conflicts or unsupported claims become Red.

7. Limitations and Future Work

The current system has several limitations:

1. It mainly relies on metadata, abstracts, and snippets from public APIs, and cannot guarantee access to full text for all cited papers.
2. Cross-disciplinary claim-support relations are sometimes difficult to determine from abstracts alone.
3. The official DeepScientist workflow is not always time-stable and may take a long time to generate claims.
4. Semantic Scholar and arXiv may rate-limit requests.
5. PDF text extraction is still fragile for scanned PDFs, two-column ordering errors, and complex layouts.
6. The number of Yellow labels is high, which shows that the system is conservative but also needs stronger automatic support checking.

Future improvements include:

- ? downloading cited paper full text and extracting paragraph-level supporting sentences;
- ? stronger version merging between arXiv preprints and journal publications;
- ? more fine-grained error types such as invalid DOI, author mismatch, claim too strong, and provider disagreement;
- ? a more stable cache to reduce repeated API calls;
- ? improved PDF layout parsing;
- ? packaging QQ demonstration artifacts into a more readable HTML summary.

8. Team Contributions

The team contribution breakdown is:

Member	Main Contribution
Guo Yaoyuan	Multi-reviewer module, local workflow testing, reviewer output, and scoring-logic debugging
Qi Jianan	Memory module, historical citation/provider/reviewer/failure records and lookup
Liu Yannan	Official DeepScientist integration, self-written skills, citation verifier, tool-use toolchain, QQ Bot, test-case organization, report, poster, PPT, and submission materials

Overall, DeepScientist generates scientific hypotheses, while the added module in this project performs evidence-chain tracking and Green / Yellow / Red audit for citation-backed claims. The system can show tool-call logs, retrieved literature, citation verification tables, evidence-chain files, QQ interaction, success cases, and boundary cases. It therefore satisfies the Project A requirement for a verifiable scientific-agent extension.

9. Reproduction and Submission Materials

The project can be run from the repository root. A typical command is:

```
python scripts/run_official_ds_case.py `
--case-id qq_pdf_demo `
--pdf "C:\path\paper.pdf" `
--run-id qq_official_demo
```

The QQ demo command is:

```
/official "C:\path\paper.pdf"
```

After each successful run, the following files should be checked:

```
citation_audit_claims.json
citation_verification.csv
evidence_chain.csv
tool_calls.jsonl
evidence_items.jsonl
deepscientist_audit_summary.md
multi_review_report.md
final_case_report.md
```

Together, these files form a reproducible citation evidence chain. If the live network or API fails during the presentation, the prepared run directory, CSV files, logs, and reports can still be used to explain the system behavior.